

| STAT214 Chapter 7 Clustering

| 1 Understanding Clusters

| 1.1 Clustering Problem

The activity of organizing the observations in your data into groups is called clustering

| 1.2 Cluster

- a cluster in a dataset is a group of observations that are similar to one another
- The definition of “similarity” depends on the context or situation.

| 1.3 Evaluating Clusters

We can quantify the quality of a set of clusters based on cluster properties including:

- **Cluster tightness**: How similar (or close together) points *within* each cluster are.
- **Cluster distinctness**: How similar (or close together) points in *different* clusters are.
- **Cluster predictability**: Whether the clusters reappear in new/external data and are capturing meaningful groups in the context of domain knowledge.
- **Cluster stability**: Whether the clusters are stable across reasonable perturbations of the data, judgment calls, and algorithms.

| 1.4 Defining Similarity

- The most common similarity measures for clustering are based on distance metrics:
 - Euclidean distance
 - Manhattan distance
- If you want each variable to have equal influence on the distance calculation and the variables are not already on the same scale, then it is often a good idea to scale the variables
 - dividing each variable by its standard deviation (SD)
 - converting them to a common range

| 1.5 The Manhattan (L1) Distance

The Manhattan, or L1, distance between two observations is a distance metric that is based on the absolute value difference between the measurements across each variable for the two observations.

$$d_1(x_1, x_2) = \sum_{j=1}^p |x_{1,j} - x_{2,j}|$$

| 1.6 The Euclidean (L2) Distance

The Euclidean, or L2, distance between two observations is a distance metric that is based on the squared difference between each measurement across each variable for the two observations.

$$d_2(x_1, x_2) = \sqrt{\sum_{j=1}^p (x_{1,j} - x_{2,j})^2}$$

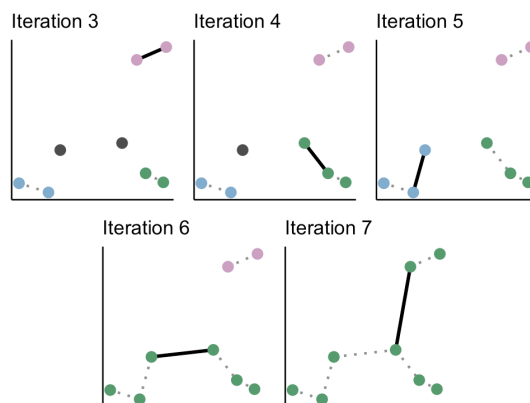
| 2 Hierarchical Clustering

| 2.1 Hierarchical Clustering

The hierarchical clustering algorithm

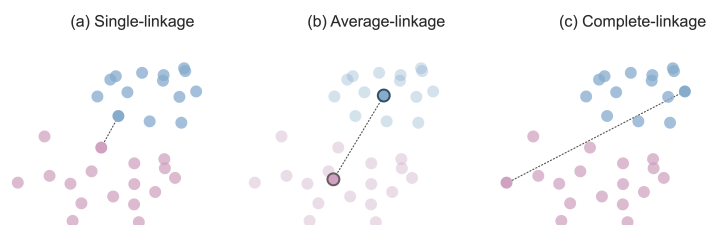
- starts by placing every observation in its own cluster

- then **iteratively merges the two most similar clusters** based on a cluster similarity measure.



2.2 Defining Intercluster Similarity (Linkage Methods)

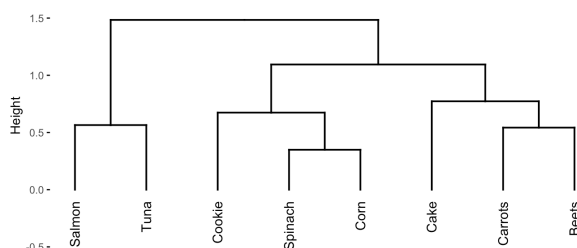
- **Single-linkage hierarchical clustering**: based on the similarity between the two most similar points from each cluster
- **Average-linkage hierarchical clustering**: based on the similarity between the two average points from each cluster
- **Complete-linkage hierarchical clustering**: based on the similarity between the two most dissimilar points from each cluster
- **Ward's linkage hierarchical clustering**: merging the two clusters whose merger will minimize the total within-cluster variance, which is defined as the sum of the variances of the data points within each cluster



2.3 Dendrogram

To help identify some interesting intermediate clusters we can generate a hierarchical clustering dendrogram

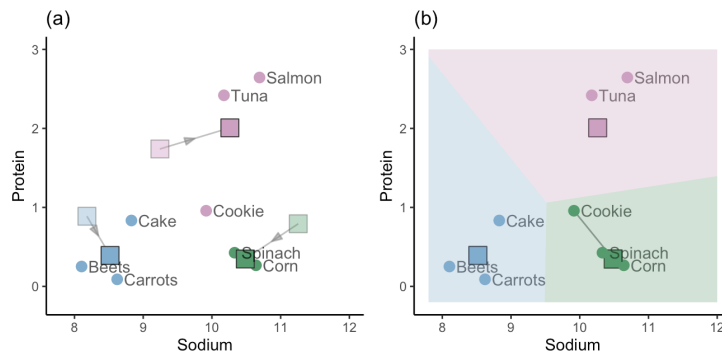
A **dendrogram** is a visual representation of the sequence of cluster merges (linkages).



3 K-Means Clustering

3.1 K-Means Clustering

1. Randomly place initial cluster centers within the data space.
2. Assign the cluster membership of each data point based on the initial cluster center it is most similar/closest to.
3. Calculate new cluster centers for each cluster based on the average of all the points that were assigned to the corresponding cluster.
4. Redefine the cluster membership of each point based on the new cluster centers.
5. Repeat steps 3 and 4 until the clusters stop changing (i.e., until the algorithm has converged).



⚠ Remark: Convergence ▾

There is no guarantee that the K-means algorithm will converge on the same set of clusters every time. The final results will still be dependent on the random placement of the *initial* cluster centers.

3.2 The K-Means Objective Function

The loss function is

$$\sum_{k=1}^K \sum_{i \in C_k} d(x_i, c_k)$$

It is standard in the K-means algorithm to use the (squared) Euclidean distance metric so this loss function can be written as

$$\sum_{k=1}^K \sum_{i \in C_k} \sum_{j=1}^p (x_{i,j} - c_{k,j})^2$$

However, although the K-means algorithm *aims* to minimize this objective function, that doesn't mean that it actually *manages* to minimize it every time. The K-means algorithm only ever finds an approximate solution to this loss function.

3.3 Preprocessing: Transformation

- Although neither the K-means nor the hierarchical clustering algorithms require Gaussian or symmetric variables, **transforming skewed variables (e.g., using a log transformation)** can often lead to more meaningful clusters if the transformation improves the symmetry of the variable distribution.
- mean-centering has no impact on the clusters

4 Visualizing Clusters in High Dimensions

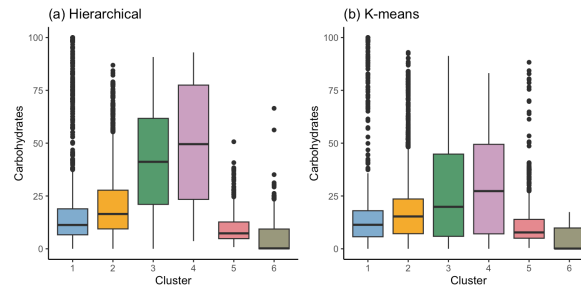
4.1 Sampling Data Points from Each Cluster

One way to visualize a set of clusters is to look at a sample of the data points that ended up in each cluster.

1	2	3	4	5	6
Strawberries	Chicken	Veal	Cheese	Collards	Sardines
Jelly	Tuna	Mayonnaise	Cornmeal	Squash	Octopus
Energy	Chocolate	Spinach	Cheese	Spinach	Herring
Custard	Egg	Tortilla	Graham	Pasta	Fish
Jelly	Bread	Caramel	Coffee	Broccoli	Octopus
Fruit	Bread	Pastry	Cheese	Steak	Salmon
Tea	Salisbury	Cereal	Yogurt	Mustard	Salmon
Prunes	Corn	Frankfurter	Ice	Carrots	Roe
Coffee	Spanish	Seasoned	Ice	Okra	Porgy
Jagerbomb	Strawberry	Egg	Ice	Artichoke	Herring
Gelatin	Bread	Kung	Cheese	Carrots	Salmon
Plum	Tuna	Cheeseburger	Fudge	Cauliflower	Ray
Apple	Infant	Hamburger	Honey	Corn	Sardines
Cherries	Chocolate	Frankfurter	Coffee	Fruit	Trout
Soup	Infant	Beef	Cheese	Rice	Porgy

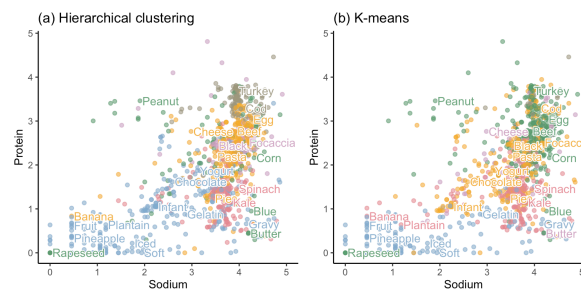
4.2 Comparing Variable Distributions across Each Cluster

Another way to visualize the clusters is to visualize the distribution of the individual variables for the observations in each cluster.



4.3 Projecting Clusters onto a Two-Dimensional Scatterplot

we can visualize them in a two-dimensional scatterplot by using the cluster membership to color each data point.



5 Quantitative Measures of Cluster Quality

5.1 Within-Cluster Sum of Squares (WSS)

The WSS for a set of clusters quantifies cluster tightness by adding together the squared Euclidean distances from each observation to its cluster center.

A general mathematical formula for computing the WSS for a set of K clusters in a dataset with p variables is given by

$$WSS = \sum_{k=1}^K \sum_{i \in C} \sum_{j=1}^p (x_{i,j} - c_{k,j})^2$$

5.2 Properties of WSS

- its value will generally become smaller as you compute more and more clusters (*WSS is not suitable for comparing the quality of sets of clusters that contain different numbers of clusters*)
- for a fixed , it will generally become larger as the number of data points increases (*WSS is not an appropriate measure for comparing clusters computed on datasets with different numbers of observations*)

⚠ Remark: Scale WSS ↘

The WSS can be made comparable across datasets with different numbers of observations by scaling the WSS by the number of observations involved in its computation.

5.3 The Silhouette Score

Define:

- a_i : The average distance between the i th data point and the other points in the *same* cluster as i .
- b_i : The average distance between the i th data point and the points in the *nearest adjacent* cluster.

Then the Silhouette scores is defined as

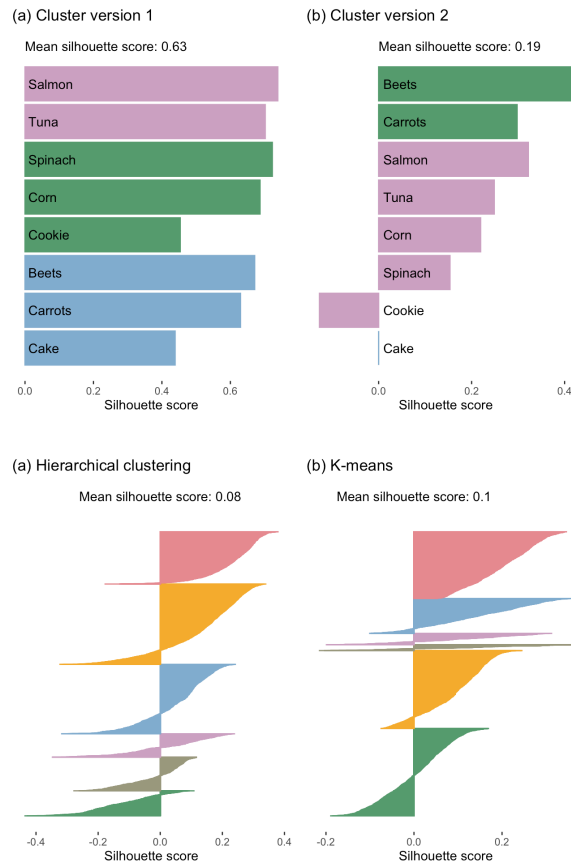
$$\text{silhouette score}_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

A data point that has a silhouette score of approximately:

- 1 is well clustered since it is much closer to the data points in its own cluster than it is to data points in other clusters.
- 0 lies between clusters since it is equally close to the data points in its own cluster and in another cluster.
- -1 is poorly clustered since it is further from the points in its own cluster than it is to points in another cluster.

5.4 The Silhouette Plot

Silhouette scores are often visualized using a **silhouette plot**, where the scores are plotted in descending order as horizontal bars (one for each observation) grouped by cluster.



6 The Rand Index for Comparing Cluster Similarity

6.1 Rand Index

The Rand Index is a measure that compares two possible ways of clustering a particular collection of data points in terms of how similarly the data points are clustered in each.

$$\text{Rand Index} = \frac{\# \text{ pairs in agreement}}{\# \text{ total pairs}}$$

6.2 The Adjusted Rand Index

An alternative to the Rand Index is the **adjusted Rand Index**, which

- lies between -1 and 1
- has been “corrected for chance”

$$\text{Adjust Rand Index} = \frac{\text{Rand Index} - \text{expected Rand Index}}{\max(\text{Rand Index}) - \text{expected Rand Index}}$$

7 Choosing the Number of Clusters

7.1 Using Cross-Validation to Choose K

V -fold CV aims to emulate the training/validation split for evaluating results on “pseudo”-validation data. The general process is as follows:

1. Split the data into V equal-sized non-overlapping subsets, called *folds*.
2. Remove the first fold (this fold will play the role of the pseudo-validation set), and use the remaining $V - 1$ folds (the pseudo-training set) to train the algorithm.
3. Use the withheld first fold (the pseudo-validation set) to evaluate the algorithm that you just trained on the other $V - 1$ folds using a relevant performance measure.
4. Replace the first fold, and remove the second fold (the second fold is now the pseudo-validation set). Train the algorithm using the other $V - 1$ folds (including the previously withheld first fold). Evaluate the algorithm on the withheld second fold.
5. Repeat step 4 until each fold has been used as the pseudo-validation set, and you have V values of the performance measure for your algorithm.
6. Combine (e.g., compute the mean of) the V performance measure values, each computed on a withheld fold.

The split into training and validation sets (and test set, where relevant) should be informed by how the results will be applied to *future data* (e.g., using a time-based, group-based, or random split).

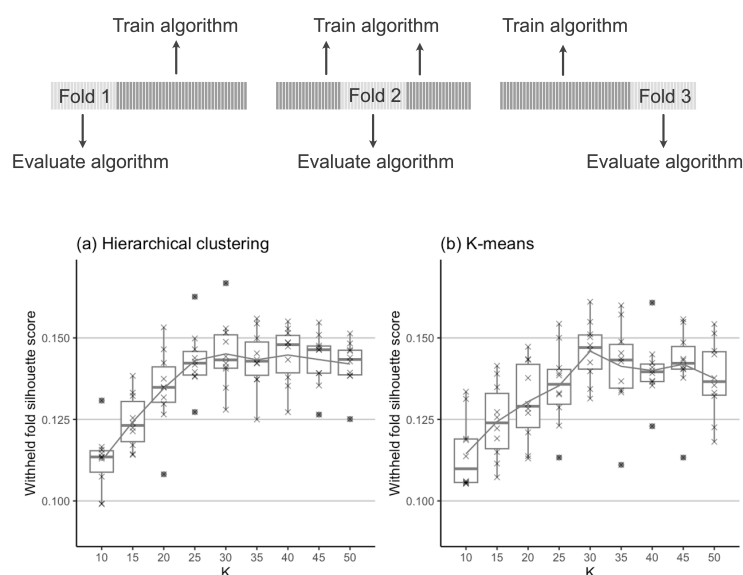


Figure 7.24: Boxplots demonstrating the distribution of CV silhouette scores for values of K (number of clusters) ranging from 10 to 50 by intervals of 5 for (a) the hierarchical clustering algorithm (with Ward linkage), and (b) the K-means algorithm. A line displaying the mean of the average silhouette scores for each K is shown, and each individual withheld-fold silhouette score is shown as an “times”.

⚠ Remark

While CV provides a great lightweight PCS analysis for hyperparameter selection, we recommend that you still perform a detailed PCS evaluation of your final results.

7.2 Number of Folds

- 5-fold CV (i.e., $V = 5$) is common for small datasets (i.e., datasets with up to a few thousand data points)
- 10-fold ($V = 10$) CV is common for larger datasets.
- The extreme case where you let the number of folds, V , be equal to the number of observations (i.e., each observation is its own fold) is called **leave-one-out cross-validation**.

8 PCS Scrutinization of Cluster Results

8.1 Predictability

Evaluate the predictability by considering how well an external dataset get clustered when their cluster membership is determined based on which of the original cluster centers it is closest to.

- compare the WSS (we may need to divide the WSS by the number items in dataset)
- see whether some randomly chosen newly clustered items match the categories that we identified before.

8.2 Stability

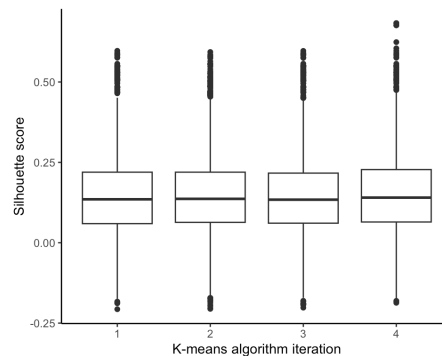
8.2.1 Stability to the Choice of Algorithm

Compare using the K-means algorithm versus the hierarchical clustering algorithm:

- average silhouette score (distribution of the silhouette score)
- (Adjusted) Rand Index
- whether the clusters identified are similar

8.2.2 Stability to Algorithmic Randomness

- compared the distribution of the silhouette score
- (Adjusted) Rand Index
- whether the clusters identified are similar



8.2.3 Stability to Data Perturbations

Create perturbed versions of the data, where the perturbations each correspond to

- a bootstrap sample of the original dataset
- adding a small amount of random noise to the nutrient measurements

then compare

- average silhouette score (distribution of the silhouette score)
- (Adjusted) Rand Index

8.2.4 Stability to Preprocessing Judgment Call Perturbations

Since SD-scaling is necessary to ensure that variables with larger values do not dominate the distance calculations, we will focus on exploring how our results change when we log-transform *all* the variables versus *none* of the variables.

then compare

- average silhouette score (distribution of the silhouette score)
- number of items in each cluster

